

ECE4760 Lab 1: Synthesizing Birdsong Report

Stephen Barlett, Jamayne Gyimah-Danquah, Md Shad

September 24, 2025

1 Introduction

In Lab 1, we built a birdsong synthesizer on the RP2040 microcontroller to recreate the call of the Northern Cardinal. The song was broken down into basic primitives such as chirps, swoops, and silences, which we generated using direct digital synthesis (DDS). We utilized a sine lookup table, combined with fixed-point arithmetic and numerical integration, to produce smooth frequency sweeps. The samples were then sent to a DAC over SPI to create the analog sound, with amplitude envelopes applied to remove clicks and make the transitions sound natural and closer to that of a Northern Cardinal.

We also used a keypad for user control, allowing individual sounds to be played or sequences to be recorded and replayed. Debouncing was handled with a finite state machine, and GPIO pins were used for both input and timing measurements. Timer interrupts ensured precise synthesis timing, while concurrency allowed sound generation and keypad input to run without interfering with each other. To closely match the real birdcall, we applied curve-fitting to spectrogram data and characterized our code to optimize ISR performance. The result was a convincing digital reproduction of the Northern Cardinal's song that demonstrated both embedded systems programming and digital signal processing techniques.

2 Design and Testing

2.1 Overview

To accomplish producing the Northern Cardinal birdsong we split up the tasks into three weeks. During Week 1, we focused on establishing the foundational hardware and verifying basic direct digital synthesis. This involved connecting the DAC to the RP2040, generating a simple sine wave “beep,” observing the waveform on an oscilloscope, and routing the audio output to a speaker. We also tested sending the waveform to the second DAC channel and added the boot select button to the breadboard.

In Week 2, we integrated the keypad with the DAC output, making sure we were producing proper key detection and implementing debouncing. The program was modified so that pressing the 1 key produced a swoop sound and the 2 key produced a chirp, both with amplitude modulation to avoid clicks or pops. We also measured the ISR execution time

using a GPIO pin and oscilloscope and explored optimizations to reduce computation time for longer sound primitives.

Week 3 expanded the system to support two modes: play and record. In play mode, the device generated the swoop and chirp sounds on keypresses, while in record mode it stored the sequence of inputs and played them back accurately when switching back to play mode. Throughout, we verified timing, sound quality, and ISR performance to ensure reliable operation.

2.2 Direct Digital Synthesis

To generate precise audio tones for our bird song for the Northern Cardinal, we implemented Direct Digital Synthesis (DDS), a technique that converts a frequency-time function into a real-time stream of digital samples. Instead of storing full waveforms for each sound segment, our system uses a 32-bit phase accumulator and a 256-entry sine lookup table to calculate each sample efficiently on the fly. The swoop is modeled with a sine function, producing a smooth oscillating tone, while the chirp is implemented as a quadratic frequency ramp that rapidly increases from low to high frequency. Silence is represented by zero amplitude.

Swoop:

$$f[n] = 1740 + 260 \cdot \sin\left(\pi \frac{n}{6499}\right) \text{ Hz}, \quad n = 0, 1, 2, \dots, 6499$$

Chirp:

$$f[n] = 2200 + 0.0001183 \cdot n^2 \text{ Hz}, \quad n = 0, 1, 2, \dots, 6499$$

Silence:

$$f[n] = 0 \text{ Hz}, \quad n = 0, 1, 2, \dots, N - 1$$

Figure 1: Defining functions for sound primitives.

Each segment lasts 130 milliseconds, and during playback or recording, the interrupt service routine computes the correct sample for the current phase, applies an amplitude envelope with attack and decay, and outputs it to the DAC. The attack and decay envelope ensures smooth amplitude transitions at the start and end of each tone, preventing clicks and artifacts in the output.

By combining DDS with precise timer-based interrupts, our system maintains a 50 kHz sample rate, ensuring accurate reproduction of frequency modulations and temporal spacing between notes. This tight control over timing, amplitude, and frequency allows our system to replicate the swoops and chirps of the Northern Cardinal while keeping memory usage minimal. Additionally, this design supports simultaneous keypad scanning, recording, and playback, as the ISR handles real-time audio generation independently of the main thread and protothreads, ensuring consistent and responsive operation.

2.3 Hardware Setup

The Bird Song Synthesizer was implemented using a Raspberry Pi RP2040 microcontroller as the central processing unit. The RP2040 executes the direct digital synthesis (DDS) algorithm, manages the timing-critical interrupt service routines, and coordinates multiple threads for handling user input, playback, and recording.

Audio output is produced through an MCP4822 12-bit digital-to-analog converter (DAC) connected via SPI, which converts the discrete waveform samples generated by the RP2040 into analog signals suitable for playback through a speaker. User interaction was facilitated through a keypad, debounced and read by the RP2040, allowing selection of sound primitives and toggling between play and record modes.

We used an oscilloscope to verify the output waveforms, measure ISR execution times, and confirm the amplitude modulation envelopes, ensuring accurate and artifact-free synthesis of the Northern Cardinal’s song.

2.4 Keypad

The keypad consists of a matrix of rows and columns that allows multiple keys to be read using a limited number of GPIO pins. Each button connects one row line to one column line. According to Figure 2, row lines are driven by digital output pins with 330 Ω current-limiting resistors, as shown for GPIO 9 through GPIO 12 in the diagram, to protect against short circuits when multiple buttons are pressed simultaneously. The column lines, connected to GPIO 13 through GPIO 15, are configured as digital inputs with internal pull-up resistors enabled so they normally read high when no key is pressed.

To detect a key press, the microcontroller sequentially drives each row low while reading the column inputs, performing this scan every 30 ms. The combination of the active row and the detected column is then matched against a lookup table in software to identify which key has been pressed. For example, pressing the “7” key closes the switch between the third row (GPIO 11) and the first column (GPIO 13); when that row is driven low, the corresponding column input is pulled low, allowing the firmware to recognize the “7.”

Even if two buttons are pressed at the same time, the firmware will register only one input during each 30 ms scan cycle because it activates only one row at a time and reads columns for that single row. Any additional pressed key in a different row will not be detected until its own row is scanned in the next cycle, preventing multiple simultaneous detections and avoiding “ghosting” effects. We also implemented debouncing so the software waits for the quick “bouncing” vibrations of the mechanical switch to settle, making sure each press is counted only once instead of several times in rapid succession.

3 Implementation

3.1 Software

We built a keypad-controlled audio system on the Raspberry Pi Pico that generates clean, low-latency tones using protothreads, timer-driven interrupts, Direct Digital Synthesis (DDS),

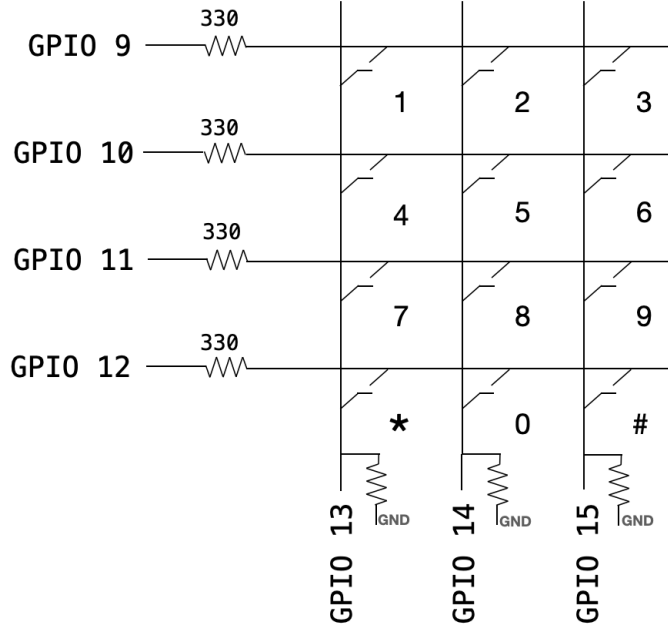


Figure 2: Keypad circuitry.

and SPI-connected DAC output. At startup the program sets the system clock, enables standard I/O, configures the keypad rows and columns, initializes SPI for a 12-bit DAC, and prepares a GPIO pin for LED heartbeat and interrupt timing. A 256-entry fixed-point sine lookup table provides the waveform source for all generated tones.

Three sound modes are implemented: a sinusoidal “swoop” with time-varying frequency, a quadratic chirp, and a period of silence. Each mode runs for a fixed number of samples that correspond to about 130 ms at a 50 kHz sample rate. DDS phase accumulation determines the instantaneous frequency for each sample. The interrupt routine, triggered every 20 microseconds, computes the next DDS increment, fetches the corresponding sine value, applies an amplitude envelope, and sends the result to the DAC through SPI.

An amplitude envelope with adjustable attack and decay times shapes every beep. During the first part of a tone the amplitude ramps smoothly upward, and near the end it ramps down. This prevents the sudden and sharp waveform edges that would otherwise produce audible clicks or pops.

Keypad input is scanned about every 30 ms in a dedicated protothread. A finite-state machine debounces the mechanical keys, confirms valid presses, and tracks releases. One key toggles between record and playback modes. Other keys either trigger sounds immediately during playback mode or are stored in a recording buffer when the system is in record mode.

Recorded key sequences are replayed in a separate protothread. This thread waits on a semaphore set when recording ends, then steps through the stored key codes one at a time. For each code it triggers the appropriate sound and waits until the interrupt routine finishes generating the beep before moving on.

Fixed-point arithmetic is used throughout the audio path for efficient digital signal processing. The design cleanly separates keypad scanning, audio triggering, interrupt-driven sample generation, and playback control. As a result the system can record and replay

Hardware Component	Purpose / Role in Lab
Raspberry Pi Pico	Served as the main microcontroller for the project, executing all code, handling keypad input, running protothreads, managing the ISR, and controlling audio output via the DAC.
Keypad (4x3)	Acted as the primary user input device. Key presses were scanned and debounced in software, allowing us to record sequences, toggle between play and record modes, and trigger audio events.
LED (GPIO 25)	Functioned as a visual heartbeat indicator to monitor program execution and thread activity in real time, helping to confirm that the system was actively scanning inputs and running the ISR.
SPI DAC (via SPI0)	Converted the digital audio samples generated by the microcontroller into analog signals for playback. This allowed precise real-time sound output for swoop, chirp, and silence modes.
GPIO pins for ISR timing (e.g., GPIO 2)	Enabled observation of ISR execution timing on an oscilloscope, allowing verification of consistent 50 kHz sample rate and accurate timing for audio generation.
Oscilloscope	Used to visualize and debug both ISR timing and DAC output waveforms, ensuring that audio signals were generated at correct frequencies and durations.
Breadboard and connecting wires	Facilitated flexible prototyping and hardware connections between the Raspberry Pi Pico, keypad, DAC, and LEDs, allowing quick adjustments during the lab.

Table 1: Hardware components and their roles in the Birdsong Synthesizer project.

key sequences, maintain a precise 50 kHz output rate, and deliver smooth, click-free audio suitable for real-time embedded applications on the RP2040.

3.2 Debouncing

Debouncing ensures that each physical press or release of a keypad button generates only one clean signal, even though the mechanical contacts inside the button naturally bounce and produce multiple rapid on-off transitions. In our code, the function `handle_key_state_machine()` implements debouncing as a state machine with four states. It begins in the *Not pressed* state (STATE_0), scanning the keypad for input. When a key is first detected, the machine moves to *Maybe pressed* (STATE_1) and stores the key in the variable `possible`. On the next scan, about 30 ms later, if the same key is still detected, the machine transitions to *Pressed* (STATE_2) and triggers a single “key pressed” event. While in this state, it continues monitoring the key, and when a mismatch or release occurs, it moves to *Maybe not*

pressed (STATE_3) to confirm the release. Only if the next scan shows no key does it return to *Not pressed*, completing a clean key-release event. By requiring two consistent readings before registering any press or release, this logic filters out the noisy bounce of mechanical switches, ensuring that each physical action on the keypad produces exactly one valid input in the code.

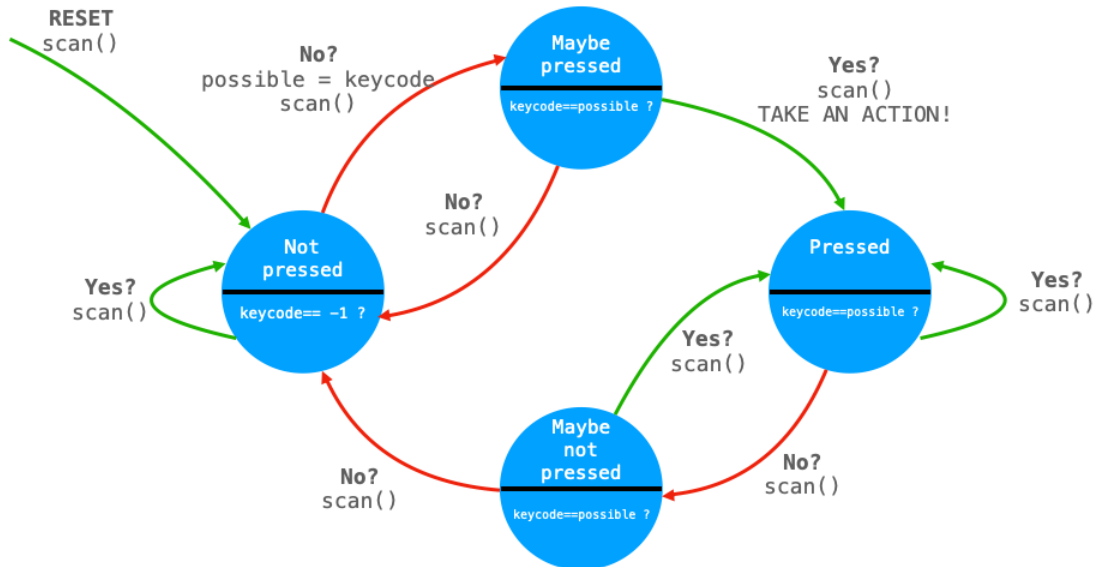


Figure 3: Debouncing FSM control flow diagram.

3.3 Recording Procedure

Our recording process begins in play mode by default, where you are allowed to freely test and play sounds without storing any sequences. We chose the star key as the control that moves the system between play and record modes. When we press the star for the first time, the program switches into record mode from play mode. In this state, we play any combination of tones using button 1 for a swoop, button 2 for a chirp, and button 3 for a period of silence. Each key press is immediately stored and the system plays the corresponding sound so we can hear it as we go.

The recorded sequence is saved in a global buffer so it stays accessible across the entire program. While we are in record mode, every press of button 1, 2, or 3 adds a code for a swoop, chirp, or silence to this global array, and a separate counter keeps track of how many entries have been stored. Because the array is global, the playback routine can later read the exact sequence in order, even though recording and playback happen in different threads and are controlled by interrupts.

Initially, we considered using either bit masking to store the sequences or dynamically allocating an array with `malloc`, but we realized that exceeding a certain length could disrupt the memory system. Therefore, we settled with a finite length for our global array which stores 50 elements. Then, if the star key is pressed again, recording stops and the system goes

to the playback state. The playback thread steps through the stored sequence, triggering the swoops, chirps, or silences each for 130 milliseconds and waiting for the interrupt routine to finish each tone before moving on. After the entire sequence has been played, the program returns to play mode on its own, ready for us to test sounds again or start a new recording session.

Overall Design/Setup

The keypad, memory, DDS synthesis, and record/play state logic work together in a tightly coordinated system to enable interactive audio generation. When a key is pressed, the keypad protothread scans the hardware, debounces the input using a finite state machine, and communicates confirmed presses to the main control logic. Depending on whether the system is in record mode or play mode, the pressed key either triggers immediate audio output through the DDS synthesizer or is stored in the `record_sequence` array in memory. During playback, the playback protothread iterates through the recorded sequence, sending each stored key code to the DDS system, which generates the corresponding tone in real time.

The DDS synthesizer operates entirely within the timer-driven interrupt service routine (ISR), calculating frequencies, applying attack and decay envelopes, and sending samples to the DAC without blocking the main thread. This separation of responsibilities allows keypad scanning, sequence memory management, and audio synthesis to operate concurrently: the keypad collects input, memory holds sequences for later use, state logic determines the system mode, and the DDS ISR produces accurate audio output, ensuring smooth, low-latency playback and comparable reproduction of recorded bird songs. The threads and ISR communicate through shared variables, flags, and semaphores, enabling synchronized operation while maintaining responsiveness even under rapid key presses or during sequence playback.

Software Component / Function	Purpose / Role in Lab
main()	Initializes system peripherals, builds the sine lookup table, computes envelope steps, sets up the timer-based ISR, and starts protothreads for keypad scanning and playback.
Protothreads (protothread_keypad & protothread_playback)	protothread_keypad handles scanning the keypad, debouncing key presses, and triggering audio events. protothread_playback manages playback of recorded sequences, synchronizing with the ISR to ensure correct timing.
Debounce FSM (handle_key_state_machine())	Implements a state machine to confirm key presses and releases, filtering out mechanical bounce. Maintains variables like STATE_1 to track the current state of each key.
scan_keypad_once()	Reads the keypad row and column signals, identifies which key (if any) is pressed, and stores it in current_key for the debounce FSM.
Direct Digital Synthesis (DDS)	Generates precise digital audio samples for each sound mode using phase accumulation and sine lookup tables, enabling real-time waveform generation.
Sine lookup table (sin_table)	Stores precomputed sine values to speed up waveform generation in DDS, allowing efficient, high-frequency audio synthesis without runtime trigonometric calculations.
Envelope computation (compute_envelope_steps())	Computes attack and decay increments for the amplitude envelope to produce smooth fades at the start and end of each beep, preventing clicks and pops.
Timer-based ISR (alarm_irq())	Executes every 20 μs (50 kHz) to generate and output audio samples in real-time, handle beep triggers, and update amplitude envelopes.
Playback semaphore (sem_playback)	Synchronizes the playback thread with recording completion, ensuring that recorded sequences are played in the correct order without overlapping sounds.
Fixed-point arithmetic (fix15)	Used for efficient, precise calculations of sine wave samples and amplitude envelopes, minimizing floating-point operations on the microcontroller.

Table 2: Software components and their roles in the lab project.

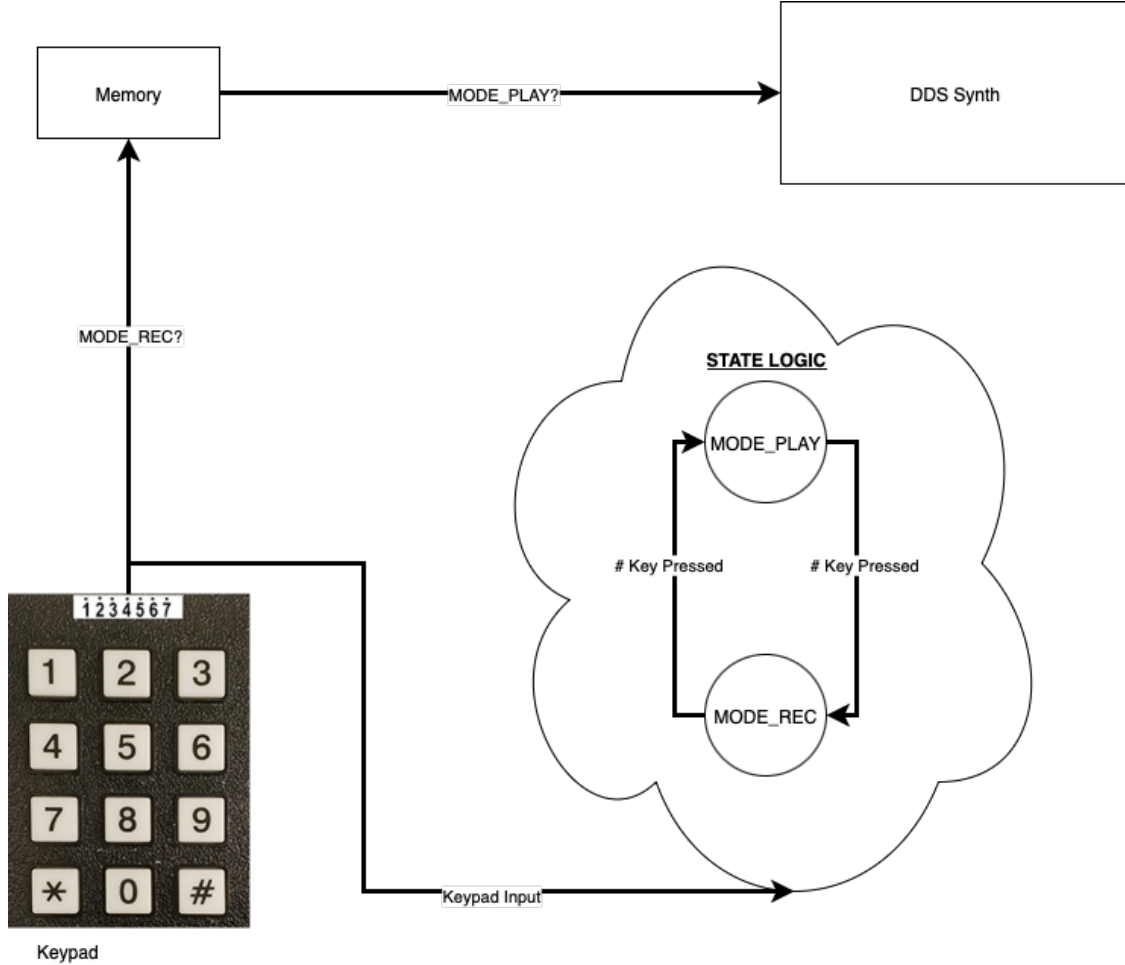


Figure 4: Playback/Record mode control flow diagram.

4 Testing

All testing for this lab was conducted through direct interaction with the microcontroller and its connected peripherals. Since the objective was to design and implement a bird song synthesizer and playback system, relatively little software-level verification was necessary; the primary requirement was that the system functioned correctly in practice.

The most valuable tool in our testing process was the oscilloscope. We used it to measure the execution time of our interrupt service routine (ISR) by setting a dedicated pin high on ISR entry and clearing it on exit. This allowed us to observe the precise time required to compute the audio waveform and perform sine table lookups (Fig. 6). Given our 50 kHz sampling rate, the ISR needed to run in less than 20 μ s. We easily met this requirement, although we later realized this was aided by the processor being unintentionally overclocked to 150 MHz (a leftover from the keypad demo code). The oscilloscope was also used to visualize the waveforms corresponding to the rise, sustain, and fall of our sound primitives (Fig. 5). By examining these shapes directly, we confirmed that the synthesizer produced the intended profiles without noticeable artifacts.

Debugging the physical hardware presented additional challenges. We relied on the lab's digital multimeter to verify electrical connections and diagnose wiring issues. At one point, keypad inputs were being read incorrectly due to a short circuit caused by two resistors making contact. While the problem was visually identifiable, we confirmed it by checking for unexpected continuity between keypad pins when no key was pressed.

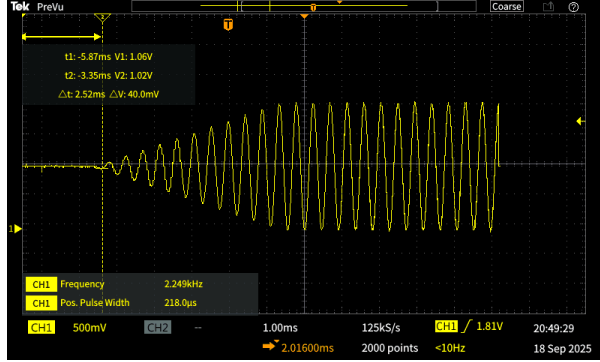
Finally, the serial monitor was an essential tool for debugging our software state machines. For example, when testing our key debouncing logic, we printed diagnostic messages that displayed the current state along with relevant input signals whenever a key press occurred. This capability provided visibility "inside" of our program and allowed us to debug logic that was not directly observable through hardware inspection.

5 AI Usage

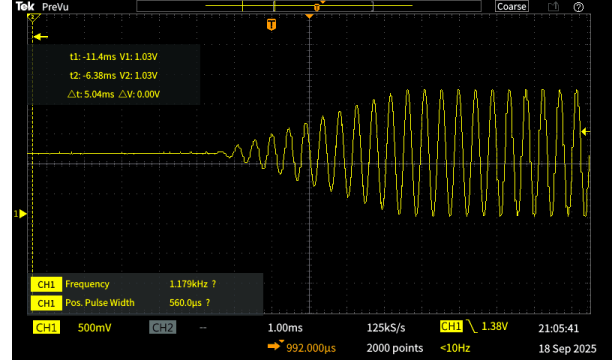
We used AI to strengthen our understanding of the code and lab expectations, in addition to guidance from Professor Adams, Professor Bruce, and the TAs during our lab sections. AI also helped us review and refresh our knowledge of C syntax and formatting, which was particularly useful for implementing the debouncing and recording functionality. However, the design, logic, and overall structure of the code were entirely our own work. We used AI primarily as a tool to verify that our syntax was correct, that our logic was sound, and to provide suggestions before we compiled and tested our code. This allowed us to focus on problem-solving and implementation while ensuring that minor syntax errors did not impede our progress. Lastly, AI was used to enhance our language and clarity of the lab report while also checking for grammatical errors along the way.

6 Documentation

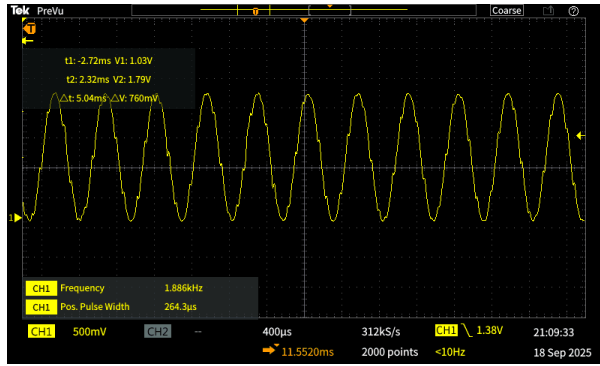
6.1 Scope of swoop/chirp rise, sustain, fall



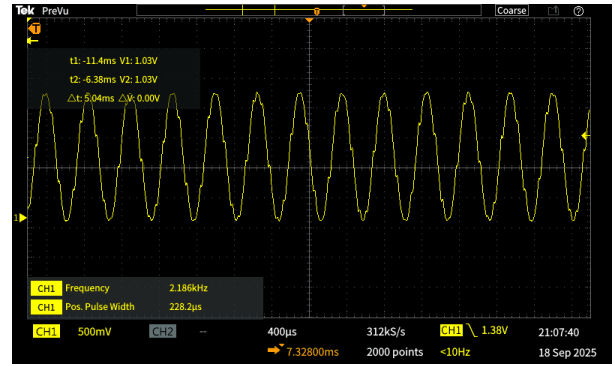
(a) Rise of swoop primitive.



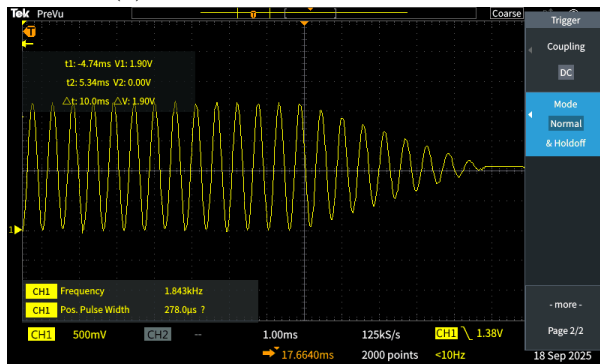
(b) Rise of chirp primitive.



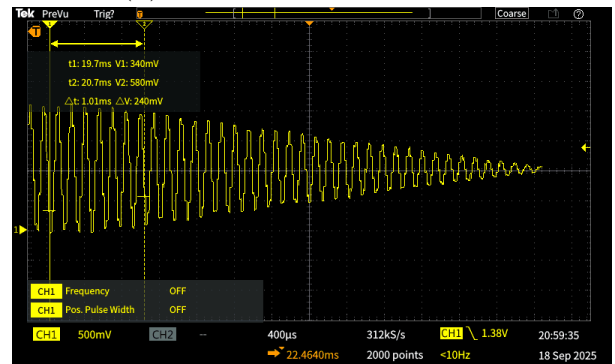
(c) Sustain of swoop primitive.



(d) Sustain of chirp primitive.



(e) Fall of swoop primitive.



(f) Fall of chirp primitive.

Figure 5: Scope traces of rise, sustain, and fall phases for both swoop (left column) and chirp (right column) primitives.

7 Results

Tricking the Merlin app proved to be a significant challenge. While the sample bird song successfully fooled the app, our synthesized output did not, even though to our untrained ears the final product sounded reasonably similar to the example. Regarding timing performance, we initially expected the swoop primitive - implemented as a sine wave - to be the most difficult to meet timing requirements. Surprisingly, even without arithmetic optimizations, timing was met fairly easily. This was achieved using the standard `sinf` C library function rather than a dedicated sine lookup table. We were able to use this function due to headroom we gained by overclocking our processor to 150MHz. In contrast, the chirp primitive, calculated using a simple quadratic expression, required significantly less computational effort and executed quickly. Overall, while achieving an exact match to the sample bird song remained elusive, the timing requirements for both primitives were successfully met under our implementation.

7.1 ISR timing comparison

To directly measure the time our MCU spends in the ISR, we set a dedicated pin high upon entry and cleared it upon exit. The resulting waveforms allow for a clear comparison of execution time between the chirp and swoop primitives. As expected, the chirp primitive executes significantly faster, reflecting its relatively simple quadratic calculation, whereas the swoop primitive, which relies on the more computationally intensive sine function, takes noticeably longer. This visualization provides a concrete demonstration of how the mathematical complexity of each primitive impacts ISR execution time.

Table 3: Time spent in ISR for each sound primitive.

Sound Primitive	Time in ISR (μs)
Swoop	10.24
Chirp	6.32

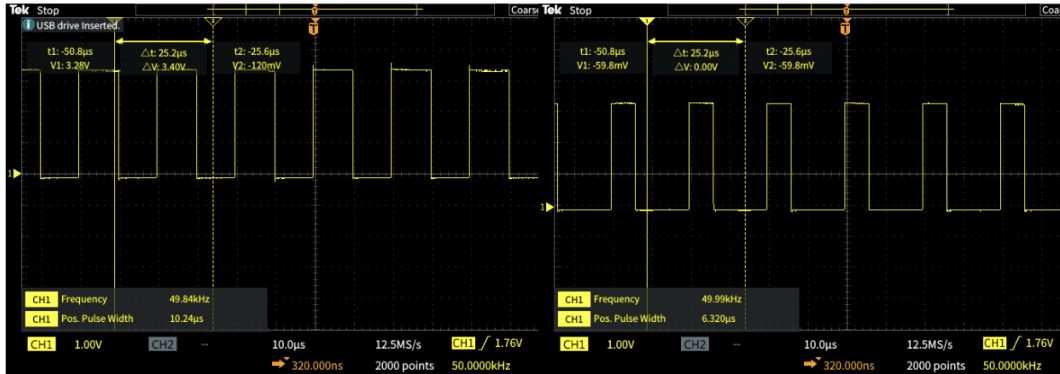


Figure 6: Comparison of ISR timing for chirp and swoop.

7.2 Soundwave Comparison

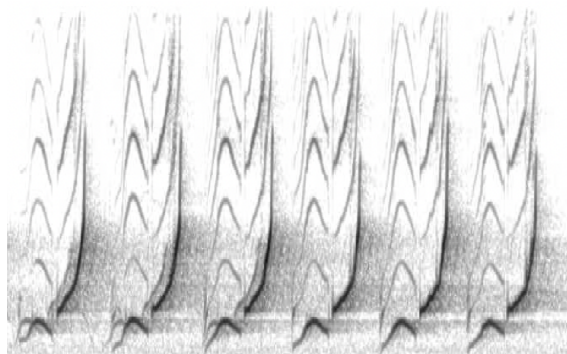


Figure 7: Spectrogram of actual northern cardinal song.

The spectrogram of the actual northern cardinal call (Fig. 7) shows a layered structure with several harmonics playing simultaneously. These frequencies appear like a shadow of the loudest frequency (the one that our program synthesizes). These overlapping harmonics make the natural birdsong much more complex than our simple DDS synth can manage.

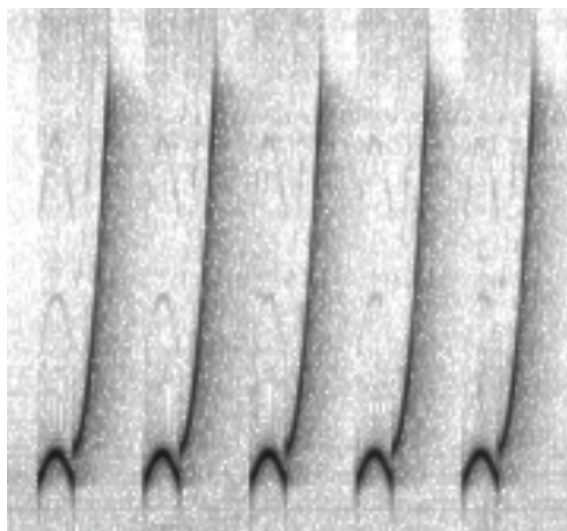


Figure 8: Spectrogram of MCU audio during playback sequence.

By contrast, the MCU-produced spectrogram (Fig. 8) shows fewer harmonics and a more artificial and precise structure. The timing of chirps and swoops aligns reasonably well with the natural call, at least from a visual perspective. However, the lack of subtle frequency layering likely contributes to Merlin being unable to identify the song.

8 Conclusion

This lab was immensely useful in helping us understand how real-time embedded systems operate when strict timing and concurrency are involved. By implementing direct digital synthesis, we were able to translate simple mathematical models into audible swoops and chirps, and we saw firsthand how applying amplitude envelopes eliminated the clicks that

appeared in earlier versions of our sounds. Working with a 50 kHz ISR taught us how tightly constrained computation time can be, and we learned that even small inefficiencies could affect the performance of the whole system. Along the way, we gained appreciation for when floating-point operations were too costly inside the ISR and how fixed-point arithmetic could be a better choice when optimizing for speed.

We also learned a lot about organizing a project across multiple threads. Setting up protothreads for keypad scanning, debouncing, and playback showed us how dividing responsibilities between threads keeps the system responsive, and we became more comfortable with context switching and synchronizing tasks using shared variables and semaphores. Debugging gave us several important lessons as well. At one point we had no sound output at all, which we traced back to not casting our sine table index to an integer before sending it to the DAC. We also had loose wires that caused keypad rows to read incorrectly, and we relied on both the serial monitor and oscilloscope to track down these issues. Using UART to print state transitions was especially helpful for verifying that our debounce state machine and record/playback logic were behaving as expected.

Even though our synthesized birdsong sounded close to the Northern Cardinal call, we were not able to fool the Merlin app. This showed us how much detail—such as additional frequency layers and timing nuances—goes into a truly natural birdsong. For future improvements, we would like to experiment with fixed-point arithmetic throughout the design and possibly add more complexity to the ISR so that multiple species or even back-and-forth bird conversations could be simulated. Overall, this lab strengthened our familiarity with the equipment, taught us how to manage timing and concurrency at a low level, and gave us confidence in debugging both hardware and software when the system did not behave as expected.

9 Code Listing

Listing 1: Main Program for birdsong synthesis and playback.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <string.h>
5 #include <stdint.h>
6
7 #include "pico/stdlib.h"
8 #include "pico/multicore.h"
9
10 #include "hardware/pio.h"
11 #include "hardware/dma.h"
12 #include "hardware/sync.h"
13 #include "hardware/spi.h"
14 #include "hardware/clocks.h"
15
16 // VGA / protothreads
17 #include "vga16_graphics_v2.h"
18 #include "pt_cornell_rp2040_v1_4.h"
19
20 #ifndef M_PI
21 #define M_PI 3.14159265358979323846
22 #endif
23
24 // ----- Pins / constants -----
25 #define BASE_KEYPAD_PIN 9
26 #define KEYROWS 4
27 #define NUMKEYS 12
28 #define LED 25
29
30 #define PIN_MISO 4
31 #define PIN_CS 5
32 #define PIN_SCK 6
33 #define PIN_MOSI 7
34 #define LDAC 8
35 #define SPI_PORT spi0
36
37 #define ALARM_NUM 0
38 #define ALARM_IRQ TIMER_IRQ_0
39
40 // Fixed-point helpers
41 typedef signed int fix15;
42 #define multfix15(a,b) (((fix15)((((signed long long)(a))*((signed long
    long)(b))))>>15))
43 #define float2fix15(a) ((fix15)((a)*32768.0f))
44 #define fix2float15(a) ((float)(a)/32768.0f)
45 #define int2fix15(a) ((fix15)((a) << 15))
46 #define fix2int15(a) ((int)((a) >> 15))
47 #define divfix(a,b) (fix15)((((signed long long)(a)) << 15) / (b))
48
49 // DDS / sample rate
```

```

50 #define two32 4294967296.0    // 2^32
51 #define Fs      50000
52 #define DELAY 20              // us (1/Fs)
53
54 // Sound modes
55 #define SOUND_SWOOP  0
56 #define SOUND_CHIRP  1
57 #define SOUND_SILENCE 2
58
59 // PLAY/REC modes
60 #define MODE_PLAY 0
61 #define MODE_REC  1
62
63 volatile unsigned int MODE = MODE_PLAY;
64 volatile bool playback_running = false;
65
66 // ISR-visible sound control (set by threads, consumed by ISR)
67 volatile int next_sound_mode = SOUND_SILENCE; // desired next sound
68 volatile bool trigger_beep   = false;        // edge to start beep
69
70
71
72
73 // ISR state (owned/cleared by ISR)
74 volatile int active_sound = SOUND_SILENCE; // current sound playing
75 volatile int beep_state   = 0;            // 0 idle, 1 playing
76 volatile unsigned int beep_duration = 0;  // samples in current beep
77 volatile unsigned int beep_count    = 0;  // samples elapsed
78
79 // Sine LUT
80 #define sine_table_size 256
81 fix15 sin_table[sine_table_size];
82
83 // ----- Swoop params -----
84 #define SWOOP_DUR_SEC 0.130f
85 #define SWOOP_NSAMPLES (int)(SWOOP_DUR_SEC * Fs)
86 static const float swoop_amp_hz = 260.0f; //Swoop Frequency Multiplier
87 static const float swoop_mid_hz = 1740.0f; //Swoop Offset Frequency
88 static const float swoop_omega  = (float)M_PI / (float)(SWOOP_NSAMPLES -
89     1); //Swoop Angle in DDS
89 volatile int swoop_i = 0;
90
91 // ----- Chirp (quadratic) params -----
92 #define CHIRP_DUR_SEC 0.130f
93 #define CHIRP_NSAMPLES (int)(CHIRP_DUR_SEC * Fs)
94 volatile int chirp_i = 0;
95 volatile float chirp_k = 1.183e-4f; // Hz per i^2
96
97 // ----- Silence params -----
98 #define SILENCE_DUR_SEC 0.130f
99 #define SILENCE_NSAMPLES (int)(SILENCE_DUR_SEC * Fs)
100 volatile int silence_i = 0;
101
102 // DAC/output

```

```

103 int DAC_output_0;
104 uint16_t DAC_data_0;
105 #define DAC_config_chan_B 0b1011000000000000
106
107 // Envelope
108 fix15 max_amplitude = int2fix15(1);
109 fix15 attack_inc, decay_inc;
110 fix15 current_amplitude_0 = 0;
111 #define ATTACK_TIME 250
112 #define DECAY_TIME 250
113
114 // Keypad
115 unsigned int keycodes[12] = {0x28,0x11,0x21,0x41,0x12,0x22,0x42,0x14,0x24,
116 ,0x44,0x18,0x48};
117 unsigned int scancodes[4] = {0x01,0x02,0x04,0x08};
118 unsigned int button = 0x70;
119 int current_key = -1;
120 volatile int possible = -1;
121 volatile unsigned int STATE_1 = 0; // debounce FSM
122
123 // GPIO used to time ISR on a scope
124 #define ISR_GPIO 2
125
126 // DDS phase accumulator
127 volatile unsigned int phase_accum_main_0;
128
129 // Recording
130 uint8_t record_sequence[50];
131 volatile uint8_t record_len = 0;
132
133 // Playback semaphore
134 static struct pt_sem sem_playback;
135
136 // ----- Forward declarations -----
137 static void init_system_clock_and_stdio(void);
138 static void init_keypad_gpio(void);
139 static void init_spi_dac(void);
140 static void init_isr_gpio_and_led(void);
141 static void compute_envelope_steps(void);
142 static void build_sine_table(void);
143 static void setup_timer_alarm_and_irq(void);
144
145 static void scan_keypad_once(int *out_current_key);
146 static void handle_key_state_machine(void);
147
148 static inline void start_beep_mode(int mode);
149 static inline void service_beep_start_if_triggered(void);
150 static inline void render_active_sample_step(void);
151 static inline void start_sound_from_code(uint8_t code);
152
153 // ----- ISR -----
154 static void alarm_irq(void)
155 {
156     gpio_put(ISR_GPIO, 1);

```

```

156
157 // Clear + re-arm
158 hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);
159 timer_hw->alarm[ALARM_NUM] = timer_hw->timerawl + DELAY;
160
161 // Check if a thread requested a beep
162 service_beep_start_if_triggered();
163
164 // Produce one sample if active
165 if (beep_state == 1)
166 {
167     render_active_sample_step();
168 }
169
170 gpio_put(ISR_GPIO, 0);
171 }
172
173 // Start-beep API the threads call
174 static inline void start_beep_mode(int mode)
175 {
176     if (beep_state != 0 || trigger_beep) return; // already playing or
177         queued
178     next_sound_mode = mode;
179     trigger_beep     = true;
180 }
181 // ----- Protothreads -----
182 static PT_THREAD (protothread_keypad(struct pt *pt))
183 {
184     PT_BEGIN(pt);
185
186     static unsigned int prev_mode;
187     prev_mode = MODE;
188
189     while (1)
190     {
191         // Toggle LED as a heartbeat
192         gpio_put(LED, !gpio_get(LED));
193
194         // Scan keypad
195         scan_keypad_once(&current_key);
196
197         // Debounce & act on transitions
198         handle_key_state_machine();
199
200         // If we just transitioned REC -> PLAY, kick playback thread
201         if (prev_mode == MODE_REC && MODE == MODE_PLAY && !playback_running) {
202             PT_SEM_SIGNAL(pt, &sem_playback);
203         }
204         prev_mode = MODE;
205
206         PT_YIELD_usec(30000); // 30 ms scan
207     }
208     PT_END(pt);

```

```

209     }
210
211 // ===== BEGIN AI-GENERATED CODE =====
212 static PT_THREAD (protothread_playback(struct pt *pt))
213 {
214     PT_BEGIN(pt);
215
216     static uint8_t i;           // survives PT_WAITS
217     static uint8_t play_len;    // loop bound snapshot (no memcpy)
218
219     while (1)
220     {
221
222         PT_SEM_WAIT(pt, &sem_playback); // Block here until someone posts
                sem_playback
223         playback_running = true; // Signal to the rest of the system that
                playback is active;
224
225         // snapshot the length ONLY (data stays in record_sequence)
226         play_len = record_len;
227         if (play_len > sizeof(record_sequence)) play_len = sizeof(
                record_sequence);
228
229         // Iterate through the frozen window [0 .. play_len-1] and add sound
                to the list
230         i = 0;
231         while (i < play_len)
232         {
233             uint8_t code = record_sequence[i];
234             start_sound_from_code(code);
235
236             PT_WAIT_UNTIL(pt, beep_state == 1); // started by ISR
237             PT_WAIT_UNTIL(pt, beep_state == 0); // finished by ISR
238
239             i++;
240         }
241
242         // clear source after playback if that's your intended UX
243         record_len = 0;
244         playback_running = false;
245     }
246
247     PT_END(pt);
248 }
249 // ===== END AI-GENERATED CODE =====
250
251 // ----- main -----
252 int main(void)
253 {
254     init_system_clock_and_stdio();
255     printf("Keypad + ISR audio (semaphore playback) starting...\n");
256
257     init_keypad_gpio();
258     init_spi_dac();

```

```

259     init_isr_gpio_and_led();
260     compute_envelope_steps();
261     build_sine_table();
262     setup_timer_alarm_and_irq();
263
264     // Init playback semaphore to 0 (blocked)
265     PT_SEM_INIT(&sem_playback, 0);
266
267     // Start threads
268     pt_add_thread(protothread_keypad);
269     pt_add_thread(protothread_playback);
270     pt_schedule_start;
271     return 0;
272 }
273
274 /*
275 AI Generated Code to put the code into static inline functions to make
276 implementation cleaner
277 */
278 // ===== BEGIN AI-GENERATED CODE =====
279
280 // ===== Helper implementations =====
281
282 static void init_system_clock_and_stdio(void)
283 {
284     set_sys_clock_khz(150000, true);
285     stdio_init_all();
286 }
287
288 static void init_keypad_gpio(void)
289 {
290     gpio_init_mask((0x7F << BASE_KEYPAD_PIN));
291     gpio_set_dir_out_masked((0xF << BASE_KEYPAD_PIN));
292     gpio_put_masked((0xF << BASE_KEYPAD_PIN), (0x0 << BASE_KEYPAD_PIN));
293     gpio_pull_down((BASE_KEYPAD_PIN + 4));
294     gpio_pull_down((BASE_KEYPAD_PIN + 5));
295     gpio_pull_down((BASE_KEYPAD_PIN + 6));
296 }
297
298 static void init_spi_dac(void)
299 {
300     spi_init(SPI_PORT, 20000000);
301     spi_set_format(SPI_PORT, 16, 0, 0, 0);
302     gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
303     gpio_set_function(PIN_SCK, GPIO_FUNC_SPI);
304     gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);
305     gpio_set_function(PIN_CS, GPIO_FUNC_SPI);
306
307     // LDAC held low
308     gpio_init(LDAC);
309     gpio_set_dir(LDAC, GPIO_OUT);
310     gpio_put(LDAC, 0);
311 }
312

```

```

313 static void init_isr_gpio_and_led(void)
314 {
315     // ISR timing pin
316     gpio_init(ISR_GPIO);
317     gpio_set_dir(ISR_GPIO, GPIO_OUT);
318     gpio_put(ISR_GPIO, 0);
319
320     // LED
321     gpio_init(LED);
322     gpio_set_dir(LED, GPIO_OUT);
323     gpio_put(LED, 0);
324 }
325
326 static void compute_envelope_steps(void)
327 {
328     attack_inc = divfix(max_amplitude, int2fix15(ATTACK_TIME));
329     decay_inc  = divfix(max_amplitude, int2fix15(DECAY_TIME));
330 }
331
332 static void build_sine_table(void)
333 {
334     for (int ii = 0; ii < sine_table_size; ii++)
335     {
336         float theta = (2.0f * (float)M_PI * (float)ii) / (float)
            sine_table_size;
337         sin_table[ii] = float2fix15(2047.0f * sinf(theta));
338     }
339 }
340
341 static void setup_timer_alarm_and_irq(void)
342 {
343     hw_set_bits(&timer_hw->inte, 1u << ALARM_NUM);
344     irq_set_exclusive_handler(ALARM_IRQ, alarm_irq);
345     irq_set_enabled(ALARM_IRQ, true);
346     timer_hw->alarm[ALARM_NUM] = timer_hw->timerawl + DELAY;
347 }
348
349 // ===== END AI-GENERATED CODE =====
350
351 // ---- keypad scan + state machine ----
352 static void scan_keypad_once(int *out_current_key)
353 {
354     static int i;
355     static uint32_t keypad;
356
357     *out_current_key = -1;
358
359     // Iterate over each row: drive one row at a time, then read columns.
360     for (i = 0; i < KEYROWS; i++)
361     {
362         gpio_put_masked((0xF << BASE_KEYPAD_PIN), (scancodes[i] <<
            BASE_KEYPAD_PIN));
363         sleep_us(1);
364         keypad = ((gpio_get_all() >> BASE_KEYPAD_PIN) & 0x7F);

```

```

365     if (keypad & button) break;
366 }
367
368 // If we saw an active column/button state during the scan...
369 if (keypad & button)
370 {
371     // Compare the sampled pattern against all known key patterns.
372     for (i = 0; i < NUMKEYS; i++)
373     {
374         if (keypad == keycodes[i])
375         {
376             // Report the key index and stop searching.
377             *out_current_key = i;
378             break;
379         }
380     }
381 }
382 }
383
384 static inline void start_sound_from_code(uint8_t code)
385 {
386     if (code == 1) start_beep_mode(SOUND_SWOOP);
387     else if (code == 2) start_beep_mode(SOUND_CHIRP);
388     else if (code == 3) start_beep_mode(SOUND_SILENCE);
389 }
390
391 static void handle_key_state_machine(void)
392 {
393     switch (STATE_1)
394     {
395     case 0: // idle
396     {
397         if (current_key != -1)
398         {
399             possible = current_key;
400             STATE_1 = 1; // maybe pressed
401         }
402         break;
403     }
404
405     case 1: // maybe pressed
406     {
407         if (current_key == possible)
408         {
409             // CONFIRMED PRESS
410             if (possible == 11)
411             {
412                 // Toggle REC <-> PLAY; REC->PLAY will signal playback in keypad
413                 // thread
414                 if (MODE == MODE_PLAY)
415                 {
416                     MODE = MODE_REC;
417                     record_len = 0;
418                     printf("\nPLAY -> REC (buffer cleared)");

```

```

418     }
419     else
420     {
421         MODE = MODE_PLAY;
422         printf("\nREC -> PLAY (will trigger playback)");
423     }
424 }
425 else if (possible == 1 || possible == 2 || possible == 3)
426 {
427     if (MODE == MODE_REC)
428     {
429         // Log key if space remains
430         if (record_len < sizeof(record_sequence))
431         {
432             record_sequence[record_len++] = (uint8_t)possible;
433             printf("\nREC idx=%u code=%d", record_len, possible);
434         }
435         else
436         {
437             printf("\nRecord buffer full");
438         }
439
440         if (beep_state == 0) {
441             start_sound_from_code((uint8_t)possible);
442         }
443     }
444     else // MODE_PLAY
445     {
446         // Directly play pressed key
447         start_sound_from_code((uint8_t)possible);
448     }
449 }
450
451 STATE_1 = 2; // pressed
452 }
453 else
454 {
455     STATE_1 = 0; // bounce -> idle
456 }
457 break;
458 }
459
460 case 2: // pressed
461 {
462     if (current_key != possible) STATE_1 = 3; // maybe released
463     break;
464 }
465
466 case 3: // maybe released
467 {
468     if (current_key == possible)
469     {
470         STATE_1 = 2; // still pressed
471     }

```

```

472     else
473     {
474         // released
475         STATE_1 = 0;
476     }
477     break;
478 }
479 }
480 }
481
482 // ---- ISR helpers ----
483 static inline void service_beep_start_if_triggered(void)
484 {
485     if (trigger_beep && beep_state == 0)
486     {
487         // Latch request, initialize envelope/counters
488         trigger_beep      = false;
489         beep_state        = 1;
490         beep_count        = 0;
491         current_amplitude_0 = 0;
492
493         active_sound = next_sound_mode;
494
495         if (active_sound == SOUND_SWOOP)
496         {
497             swoop_i      = 0;
498             beep_duration = SWOOP_NSAMPLES;
499         }
500         else if (active_sound == SOUND_CHIRP)
501         {
502             chirp_i      = 0;
503             beep_duration = CHIRP_NSAMPLES;
504         }
505         else if (active_sound == SOUND_SILENCE) // NOTE: equality, not
506             assignment
507         {
508             silence_i      = 0;
509             beep_duration = SILENCE_NSAMPLES;
510         }
511     }
512 }
513
514 static inline void render_active_sample_step(void)
515 {
516     float freq_n;
517
518     if (active_sound == SOUND_SWOOP)
519     {
520         // f[n] = f_mid + A * sin(    * n / (N-1))
521         freq_n = swoop_mid_hz + swoop_amp_hz * sinf(swoop_omega * (float)
522             swoop_i);
523     }
524     else if (active_sound == SOUND_CHIRP)
525     {

```

```

524     // Quadratic chirp: freq[n] = 2000 + chirp_k * n^2
525     float ii = (float)chirp_i;
526     freq_n = 2200.0f + chirp_k * (ii * ii);
527 }
528 else // SOUND_SILENCE
529 {
530     freq_n = 0.0f;
531 }
532
533 // DDS accumulate
534 uint32_t incr_n = (uint32_t)((freq_n * (float)two32) / (float)Fs);
535 phase_accum_main_0 += incr_n;
536
537 // LUT lookup (top 8 bits), apply envelope, center for 12-bit DAC
538 DAC_output_0 =
539     fix2int15( multfix15(current_amplitude_0,
540                          sin_table[(uint8_t)(phase_accum_main_0 >> 24)]) )
541     + 2048;
542
543 // Advance the active index for this sound
544 if (active_sound == SOUND_SWOOP)      ++swoop_i;
545 else if (active_sound == SOUND_CHIRP) ++chirp_i;
546 else                                  ++silence_i;
547
548 // Envelope (attack/decay with safe window)
549 int decay_start = (int)beep_duration - DECAY_TIME;
550 if (decay_start < 0) decay_start = 0;
551
552 if (beep_count < ATTACK_TIME)          current_amplitude_0 +=
553     attack_inc;
554 else if ((int)beep_count > decay_start) current_amplitude_0 -=
555     decay_inc;
556
557 // Write to DAC
558 DAC_data_0 = (DAC_config_chan_B | (DAC_output_0 & 0x0FFF));
559 spi_write16_blocking(SPI_PORT, &DAC_data_0, 1);
560
561 // Advance time + possibly end beep
562 ++beep_count;
563 if (beep_count >= beep_duration)
564 {
565     beep_state = 0; // ISR returns to idle (auto transition)
566     current_amplitude_0 = 0; // reset envelope
567 }
568 }

```

AI Chat Log from ChatGPT

Prompt Log Summary

Time/Date: 2025-09-24 10:32 UTC

User Input: Described a Finite State Machine (FSM) using PLAY_SEQ_REC with button

interactions.

Edits Suggested: 0 (user was stating design intention)

Code Insertions: 0

Code Deletions: 0

Code Modifications: 0

Time/Date: 2025-09-24 10:34 UTC

User Input: Posted `service_sequence_playback` function and mentioned an error.

Edits Suggested: 1

Code Insertions: 0

Code Deletions: 0

Code Modifications: 1 (pointed out likely cause of runtime error: function not called in main loop or misused)

Time/Date: 2025-09-24 10:36 UTC

User Input: Provided full code and asked to fix the playback sequence section.

Edits Suggested: 6

Code Insertions: 6

- `service_sequence_playback()` added inside protothread loop
- Non-blocking FSM transition logic inserted (REC $\rightarrow$ SEQ)
- Correct call to `start_sound_from_code()`
- Optional keypad ignore override in SEQ mode
- Proper sound mode comparison using `==`
- Single definition of `seq_play_idx` retained

Code Deletions: 2

- Old blocking for-loop for sequence playback
- Duplicate definition of `seq_play_idx`

Code Modifications: 3

- Changed FSM transition inside `handle_key_state_machine()`
- Corrected logic in sound mode assignment (SILENCE mode check)
- Updated protothread to regularly service playback